PROJECT REPORT

Google TPU Architectural Simulator

A Python-Based Performance Simulator for TPU-Inspired Architectures

Submitted by

Anoop G Krishnan (M25AI2100)
Christin Jais (M25AI2048)

April 19, 2026

Abstract

Tensor Processing Units (TPUs) are specialized accelerators designed to speed up machine learning workloads by efficiently performing matrix operations using dedicated hardware such as Matrix Multiply Units (MXUs). Modern deep learning models rely heavily on dense linear algebra, making accelerator-aware analysis important for both hardware understanding and performance optimization.

This project presents a **Google TPU Architectural Simulator**, a Python-based simulator inspired by TPU-style architectures. The simulator accepts hardware configuration files and workload descriptions as input, models the behavior of major architectural components such as the MXU, memory subsystem, vector processing unit, and roofline limits, and generates performance reports for workloads such as ResNet-50, BERT-Large, and GPT-2 Medium.

The main purpose of this project is educational and analytical. Rather than reproducing proprietary TPU internals exactly, the simulator provides a practical framework for studying how changes in architecture and workload characteristics influence compute utilization, memory traffic, arithmetic intensity, latency, and overall system behavior. The project also includes visualization support to help compare different TPU configurations and workload types through graphs.

1 Introduction

1.1 Overview

Machine learning models such as convolutional neural networks and transformer architectures are built on repeated matrix and tensor operations. These operations dominate the execution time of inference and training workloads, which motivated the development of specialized hardware accelerators.

Google TPUs are examples of such accelerators. They are designed to execute matrix-heavy workloads efficiently using a structured compute engine and optimized data movement. Understanding such hardware is valuable for students and researchers working in computer architecture, machine learning systems, and accelerator design.

This project aims to create a TPU-inspired simulator that estimates the performance of different workloads under different hardware configurations. The simulator makes it possible to analyze architectural trade-offs without requiring access to real TPU hardware.The code details are available at
https://github.com/GradientHighbury/GoogleTPUarchitecturalsimulator.git

1.2 Need for the Project

Direct hardware prototyping is expensive and time-consuming. Before real hardware is built, architects and researchers often rely on simulation to estimate expected behavior, identify bottlenecks, and compare design alternatives.

A simulator is especially useful in the context of accelerators because it allows fast experimentation with:

- Compute array sizes
- Memory bandwidth and capacity
- Workload structure
- Utilization and bottleneck trends
- Cross-generation configuration comparisons

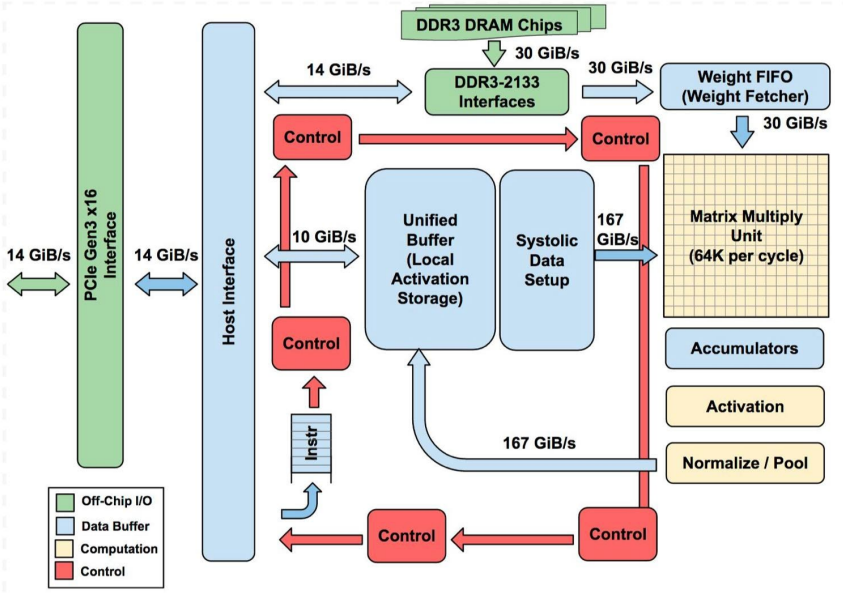

Figure 1: High-level TPU-inspired architecture overview.

2 Problem Statement

Deep learning accelerators must balance compute throughput, memory access, and data reuse. A powerful compute array alone does not guarantee good performance if memory cannot feed data fast enough. Similarly, large workloads may not fully utilize the array if their shapes are not well matched to the hardware.

The problem addressed in this project is the development of a configurable architectural simulator that can:

- Read hardware configuration files representing TPU-like architectures
- Read workload descriptions for neural network models
- Estimate performance metrics such as latency, utilization, arithmetic intensity, and compute versus memory behavior
- Generate reports and visualizations for analysis

3 Objectives

The objectives of this project are:

- To understand the architecture and design principles behind TPU-style accelerators
- To build a configurable performance simulator in Python

- To model key components such as the MXU, memory system, and vector processing unit
- To evaluate multiple workloads under multiple TPU configurations
- To generate reports and visual plots for analysis
- To provide a beginner-friendly framework for learning accelerator performance concepts

4 Background and Fundamental Concepts

4.1 What is a TPU?

A Tensor Processing Unit is a specialized processor designed to accelerate machine learning operations, especially matrix multiplication and tensor manipulation. TPUs differ from CPUs and GPUs because they are optimized for a narrower but highly important class of operations common in neural networks.

4.2 Why Matrix Operations Matter

Most deep learning layers eventually reduce to matrix multiplications, convolutions, or vector operations. Examples include:

- Fully connected layers
- Convolution layers
- Attention projections in transformers
- Feed-forward network blocks

Because these operations are repeated many times, specialized matrix hardware can significantly improve performance.

4.3 Systolic Array Concept

A systolic array is a grid of processing elements through which data moves in a rhythmic pattern. Each processing element performs a small operation, usually a multiply-accumulate step, and passes intermediate values to neighboring elements.

The main benefits of a systolic array are:

- High parallelism
- Local data reuse
- Reduced dependence on repeated memory reads
- Better efficiency for regular matrix workloads

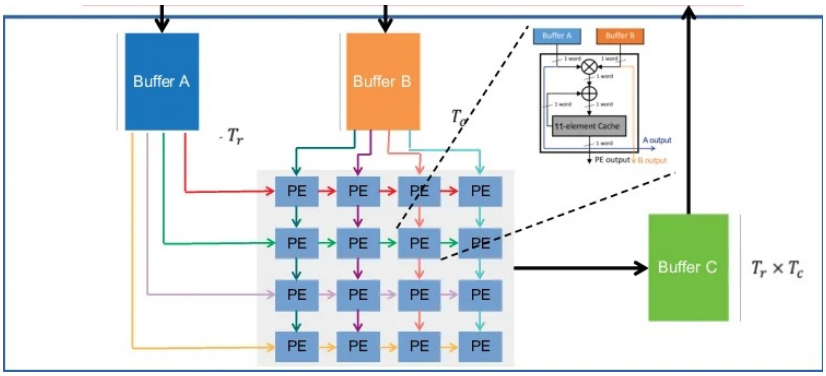

Figure 2: Conceptual systolic array used in TPU-style matrix processing.

4.4 Architectural Simulation

Architectural simulation is the software-based modeling of hardware behavior. Instead of executing a workload on real silicon, a simulator estimates what the hardware would do. This includes expected latency, utilization, memory pressure, and roofline-limited performance.

This project follows that idea. It uses workload descriptions and hardware presets to predict how a TPU-style architecture would behave for different neural network models.

5 Project Structure and Modules

5.1 Repository Structure

The project is organized as follows:

```
.
|-- configs
|   |-- tpu_v1.ini
|   |-- tpu_v2.ini
|   `-- tpu_v4.ini
|-- core
|   |-- config.py
|   |-- __init__.py
|   |-- memory.py
|   |-- mxu.py
|   |-- roofline.py
|   |-- tpu_sim.py
|   `-- vpu.py
|-- workloads
|       |-- bert_large.json
|       |-- gpt2_medium.json
|       |-- resnet50.json
|-- main.py
```

```
        |-- outputs
        |-- pyproject.toml
        |-- README.md
        |-- uv.lock
        |-- visualize.py
        `
```

5.2 Module Description

5.2.1 configs/

This folder stores the TPU hardware presets. The files `tpu_v1.ini`, `tpu_v2.ini`, and `tpu_v4.ini` represent different architectural configurations that can be simulated.

5.2.2 workloads/

This folder stores workload descriptions. These files define neural network models such as BERT-Large, GPT-2 Medium, and ResNet-50.

5.2.3 core/config.py

This module is responsible for reading and handling architecture parameters from the configuration files.

5.2.4 core/mxu.py

This module represents the Matrix Multiply Unit, which is the central compute engine of the TPU-inspired design.

5.2.5 core/memory.py

This module handles memory-related modeling. It is responsible for estimating memory movement, bandwidth effects, and other data transfer constraints.

5.2.6 core/vpu.py

This module likely models vector-oriented computations that are not handled directly by the MXU. These may include activation-like operations, reductions, or post-processing computations.

5.2.7 core/roofline.py

This module likely supports roofline-based performance analysis by comparing the arithmetic intensity of a workload against the architectural compute and memory limits.

5.2.8 core/tpu_sim.py

This module serves as the simulation engine that combines all submodules to generate performance estimates.

5.2.9 main.py

This is the command-line entry point used to run simulations.

5.2.10 visualize.py

This is the plotting script used to generate graphs from output CSV reports.

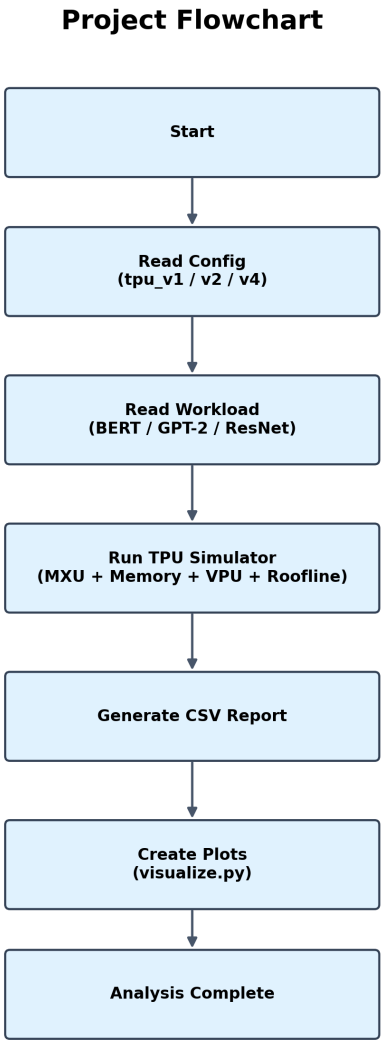

Figure 3: Overall flow of the TPU architectural simulator.

6 Working Principle of the Project

6.1 Step 1: Reading Hardware Configuration

The simulator first reads the hardware configuration file specified by the user through the command line. A configuration file defines the architectural properties of a TPU preset such as TPU v1, TPU v2, or TPU v4.

Typical configuration parameters may include:

- Compute array dimensions
- Memory properties
- Bandwidth assumptions
- Throughput-related limits
- Other architecture-specific parameters

6.2 Step 2: Reading Workload Description

The simulator then reads the workload JSON file. This file represents the model to be analyzed, such as ResNet-50, BERT-Large, or GPT-2 Medium.

The workload description contains the information needed to estimate how much computation and data movement will be required during execution.

6.3 Step 3: Mapping Work to Hardware Units

Once the workload is loaded, the simulator maps the operations to the hardware model:

- Matrix-dominant operations are handled by the MXU
- Memory transfers are handled by the memory model
- Vector or auxiliary operations are handled by the VPU

6.4 Step 4: Compute and Memory Estimation

The simulator estimates:

- How much work is sent to the matrix engine
- How efficiently the MXU is utilized
- How much data must be fetched or stored
- Whether execution is compute-dominated or memory-dominated

6.5 Step 5: Roofline Interpretation

The roofline model gives a simple visual interpretation of performance. It uses arithmetic intensity and hardware limits to determine whether a workload is limited mainly by compute capability or by memory bandwidth.

6.6 Step 6: Report Generation

After simulation, the tool writes CSV reports into the `outputs/` directory. These reports summarize the behavior of a given TPU-workload combination.

6.7 Step 7: Visualization

The `visualize.py` script reads the output reports and creates plots such as:

- Arithmetic intensity plots
- Compute versus memory plots
- Latency breakdown plots
- MXU utilization plots
- Roofline plots
- Overall comparison plots

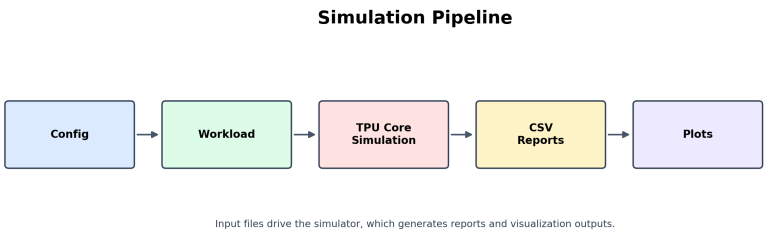

Figure 4: Step-by-step simulation pipeline from input to performance report.

7 Implementation Details

7.1 Running the Simulator

The simulator is executed from the command line by specifying both a hardware configuration file and a workload file.

```
python main.py --config configs/tpu_v1.ini --
    workload workloads/bert_large.json --outdir
    outputs
```

Other example commands are shown below:

```
python main.py --config configs/tpu_v1.ini --
   workload workloads/gpt2_medium.json --outdir
   outputs
python main.py --config configs/tpu_v1.ini --
   workload workloads/resnet50.json --outdir
   outputs

python main.py --config configs/tpu_v2.ini --
   workload workloads/bert_large.json --outdir
   outputs
python main.py --config configs/tpu_v2.ini --
   workload workloads/gpt2_medium.json --outdir
   outputs
python main.py --config configs/tpu_v2.ini --
   workload workloads/resnet50.json --outdir
   outputs

python main.py --config configs/tpu_v4.ini --
   workload workloads/bert_large.json --outdir
   outputs
python main.py --config configs/tpu_v4.ini --
   workload workloads/gpt2_medium.json --outdir
   outputs
python main.py --config configs/tpu_v4.ini --
   workload workloads/resnet50.json --outdir
   outputs
```

7.2 Generating Visualizations

To generate all plots at once:

```
python visualize.py --all --outdir outputs
```

To generate plots for a particular TPU configuration:

```
python visualize.py --config configs/tpu_v1.ini --
   outdir outputs
```

To generate plots for a specific configuration and workload:

```
python visualize.py --config configs/tpu_v1.ini --
   workload workloads/bert_large.json --outdir
   outputs
```

7.3 Expected Report Files

The project output includes report CSV files for the TPU-workload combinations, such as:

- `TPU_v1_BERT_Large_report.csv`
- `TPU_v1_GPT_2_Medium_report.csv`
- `TPU_v1_ResNet_50_report.csv`
- `TPU_v2_BERT_Large_report.csv`
- `TPU_v2_GPT_2_Medium_report.csv`
- `TPU_v2_ResNet_50_report.csv`
- `TPU_v4_BERT_Large_report.csv`
- `TPU_v4_GPT_2_Medium_report.csv`
- `TPU_v4_ResNet_50_report.csv`

8 Methodology

8.1 Input Space

The simulator studies the interaction between:

- A chosen TPU-style architecture
- A chosen workload
- A performance model composed of compute, memory, vector processing, and roofline analysis

8.2 Computation Perspective

The project assumes that matrix-heavy operations dominate execution. Therefore, the MXU is treated as the main source of computational throughput.

8.3 Memory Perspective

Even when the compute engine is powerful, performance can degrade if memory movement becomes the bottleneck. Hence the simulator separately analyzes compute demand and memory demand.

8.4 Performance Perspective

The simulator aims to answer practical questions such as:

- Which workload uses the TPU array most efficiently?
- Which TPU generation performs better for a given workload?
- Is performance limited by computation or by memory?
- How does arithmetic intensity change across models?

9 Results and Analysis

9.1 Available Output Visualizations

The output filenames indicate five plot categories for each TPU-workload pair:

- Arithmetic intensity
- Compute versus memory
- Latency breakdown
- MXU utilization
- Roofline analysis

There is also a combined comparison figure for all TPU-workload combinations.

9.2 Overall Comparison

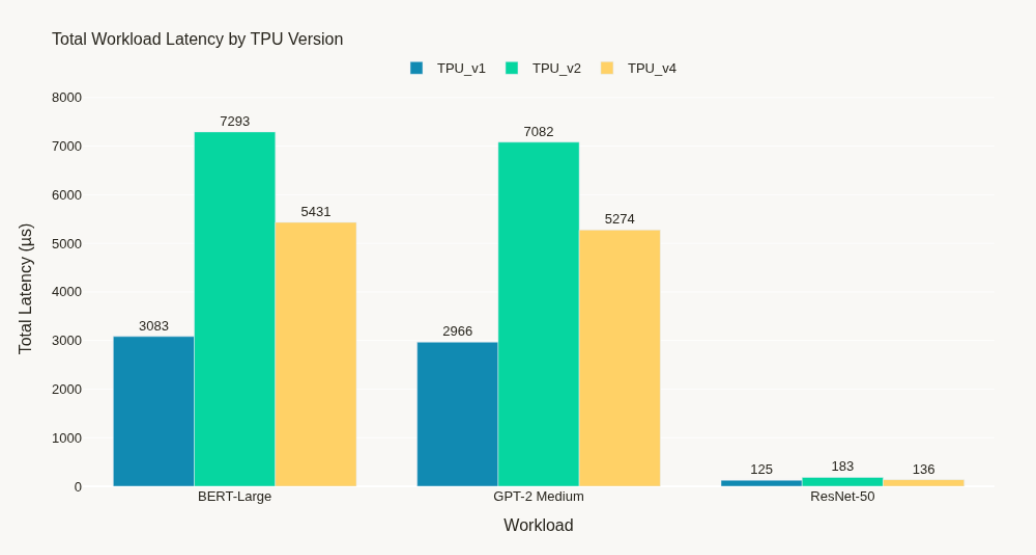

Figure 5: Overall comparison across all TPU configurations and workloads.

9.3 TPU v1 Results

9.3.1 BERT-Large on TPU v1

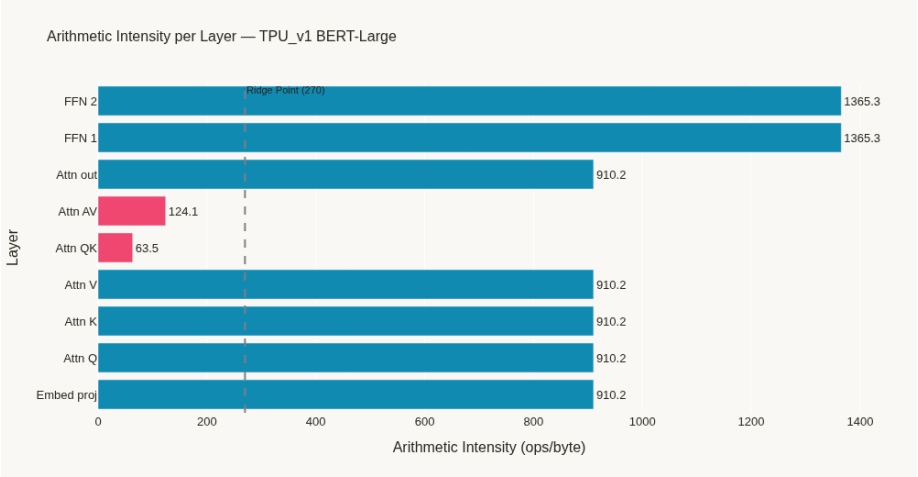

Figure 6: Arithmetic intensity for BERT-Large on TPU v1.

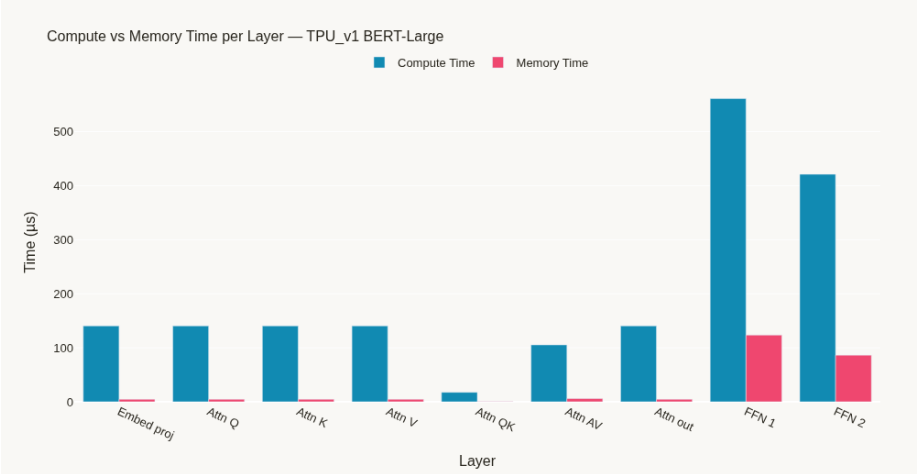

Figure 7: Compute versus memory behavior for BERT-Large on TPU v1.

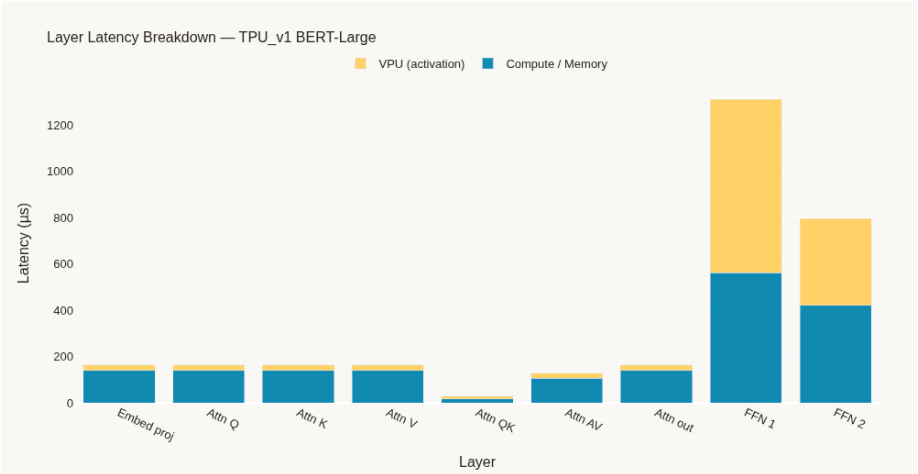

Figure 8: Latency breakdown for BERT-Large on TPU v1.

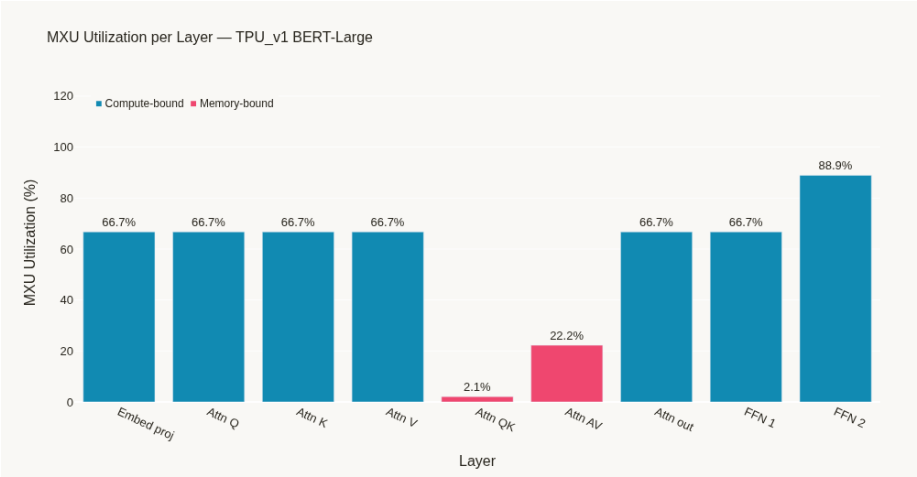

Figure 9: MXU utilization for BERT-Large on TPU v1.

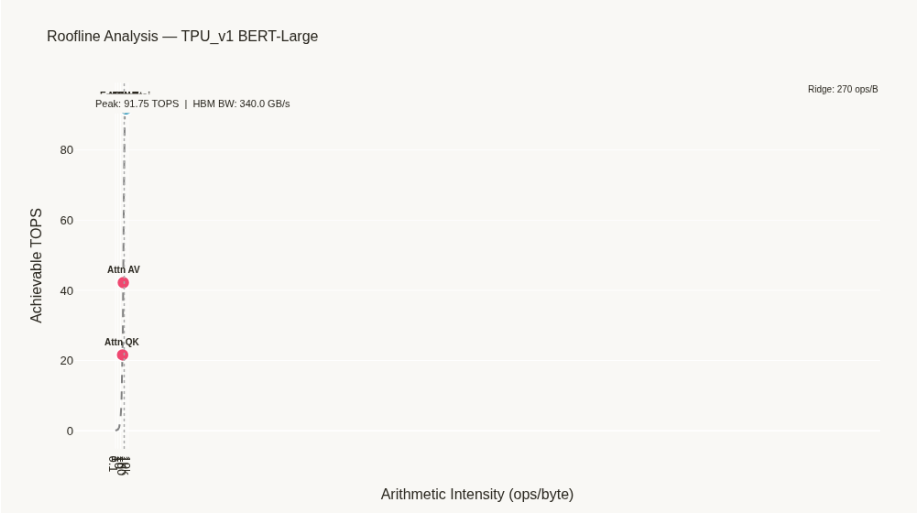

Figure 10: Roofline analysis for BERT-Large on TPU v1.

9.3.2 GPT-2 Medium on TPU v1

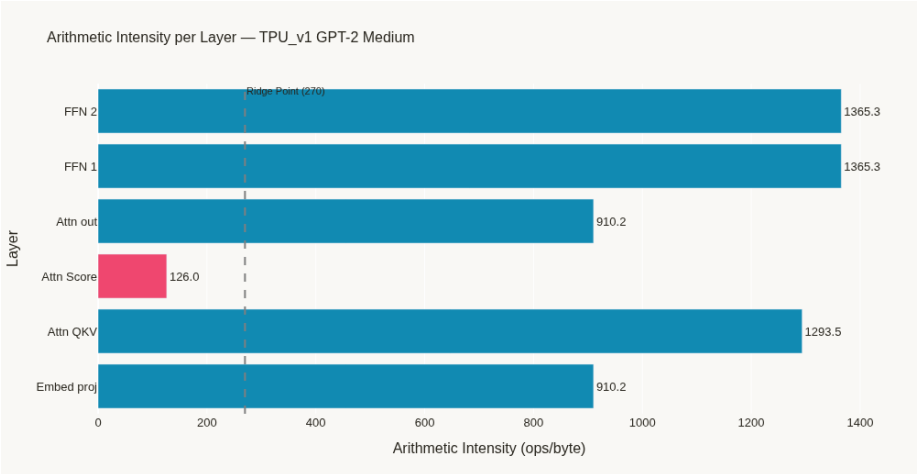

Figure 11: Arithmetic intensity for GPT-2 Medium on TPU v1.

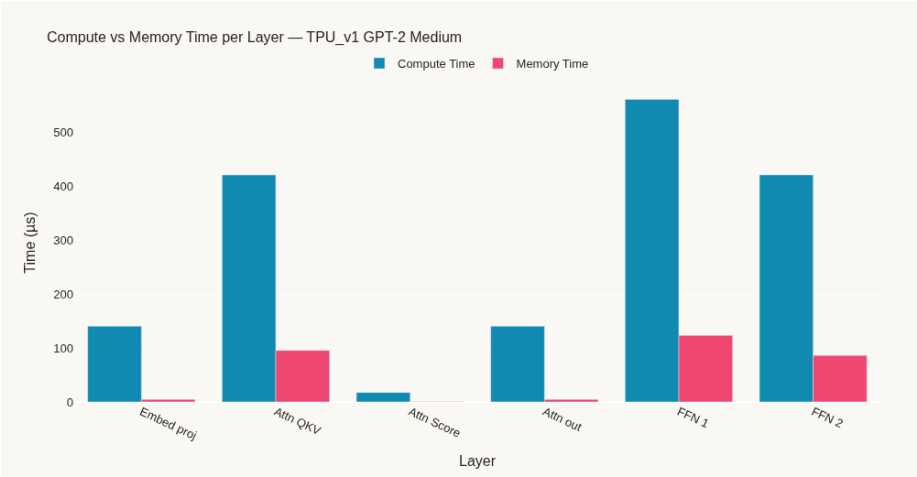

Figure 12: Compute versus memory behavior for GPT-2 Medium on TPU v1.

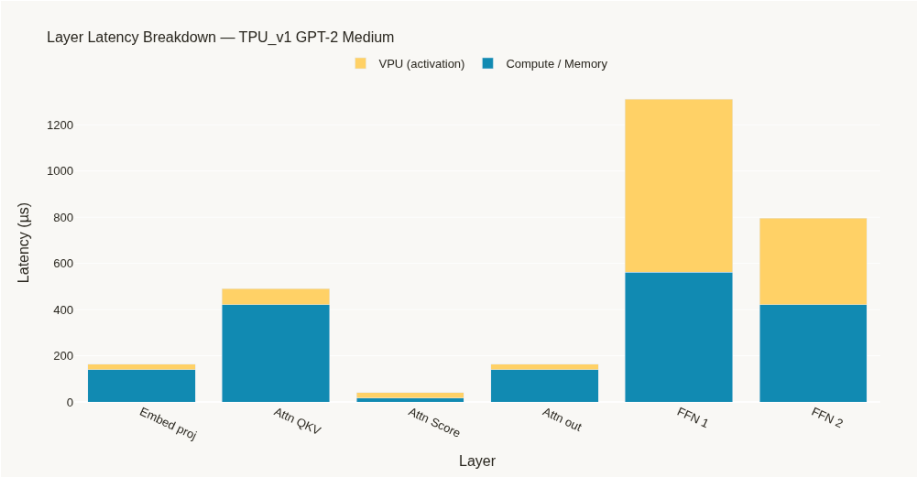

Figure 13: Latency breakdown for GPT-2 Medium on TPU v1.

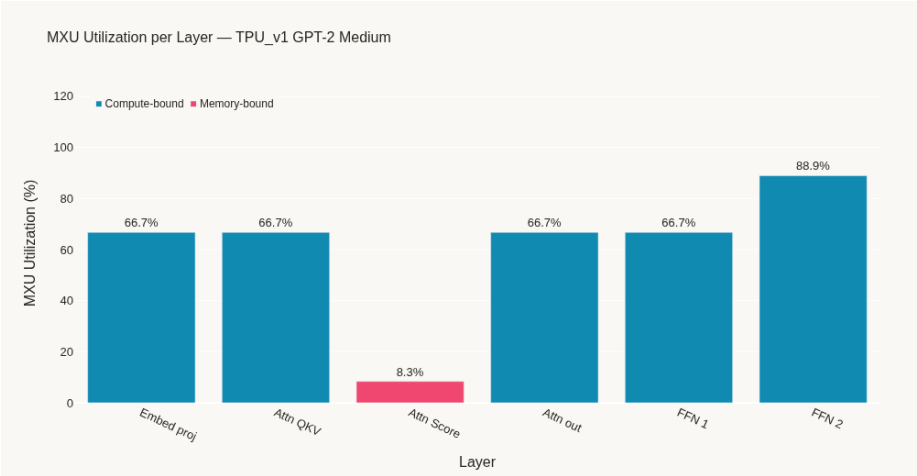

Figure 14: MXU utilization for GPT-2 Medium on TPU v1.

Figure 15: Roofline analysis for GPT-2 Medium on TPU v1.

9.3.3 ResNet-50 on TPU v1

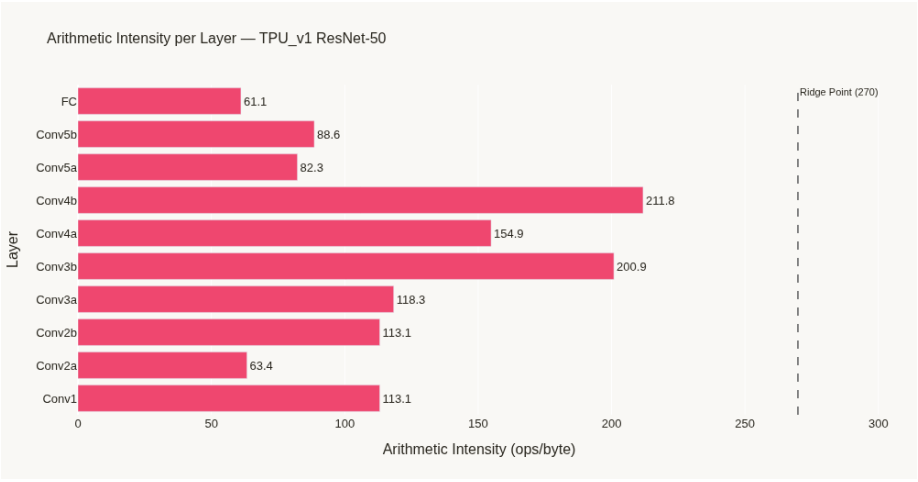

Figure 16: Arithmetic intensity for ResNet-50 on TPU v1.

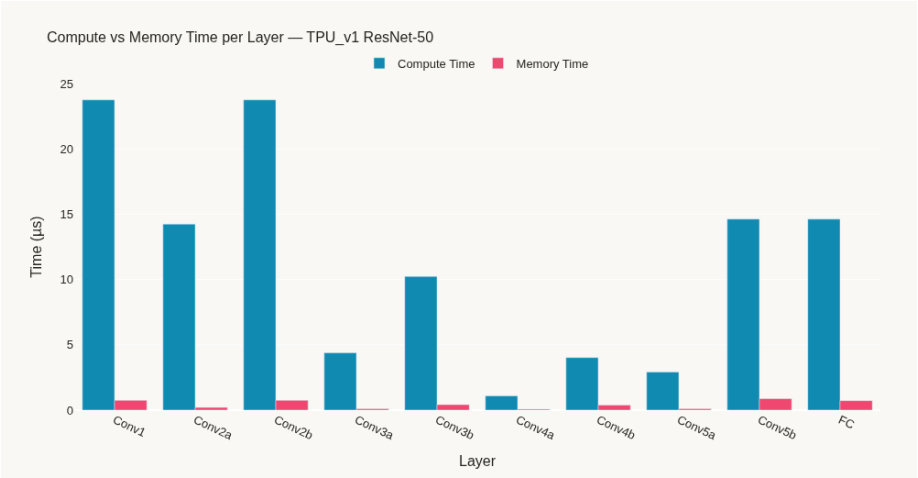

Figure 17: Compute versus memory behavior for ResNet-50 on TPU v1.

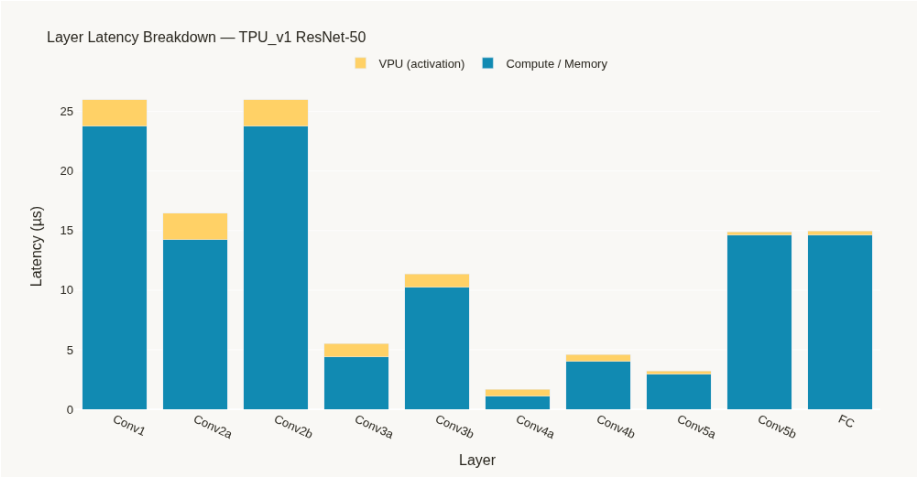

Figure 18: Latency breakdown for ResNet-50 on TPU v1.

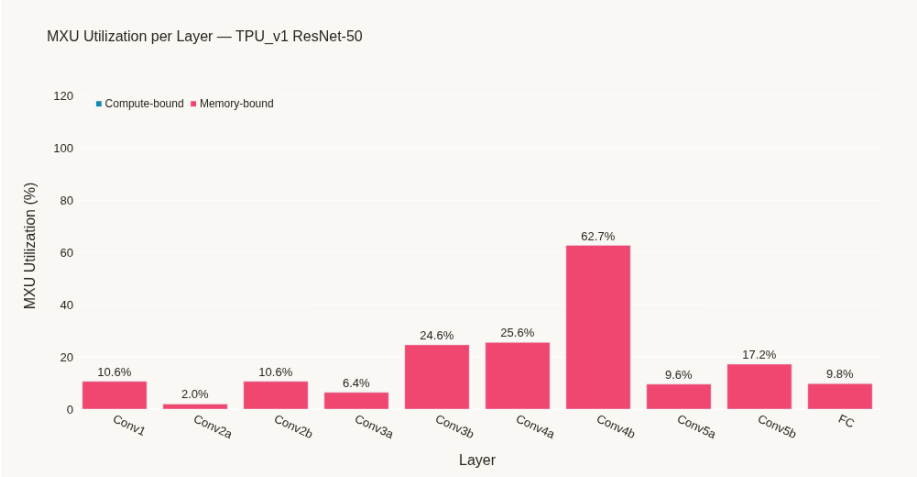

Figure 19: MXU utilization for ResNet-50 on TPU v1.

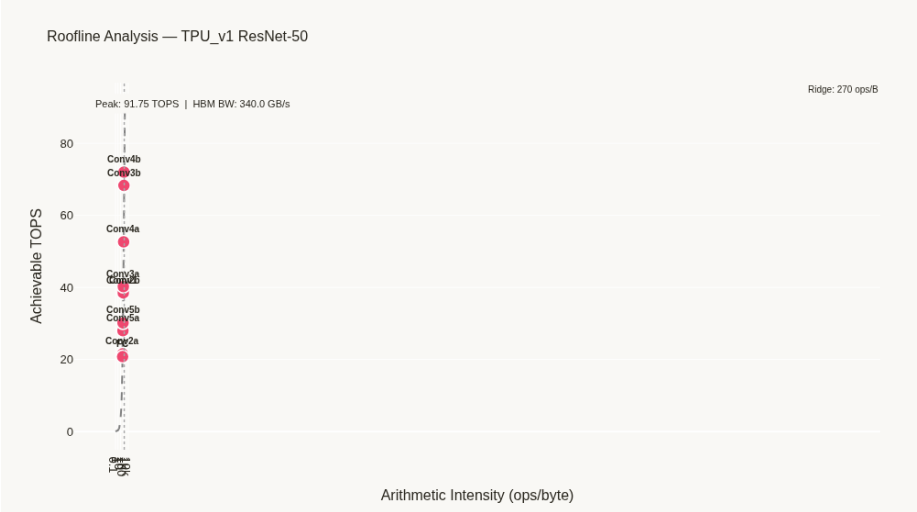

Figure 20: Roofline analysis for ResNet-50 on TPU v1.

9.4 TPU v2 Results

9.4.1 BERT-Large on TPU v2

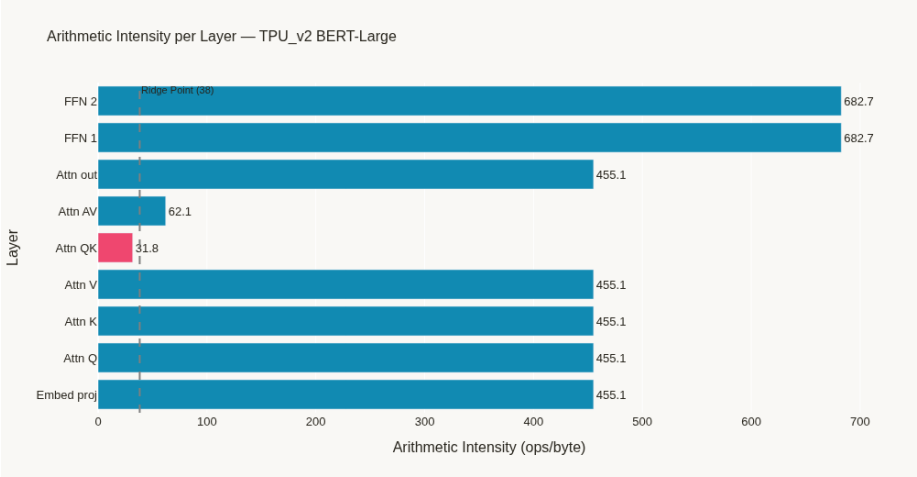

Figure 21: Arithmetic intensity for BERT-Large on TPU v2.

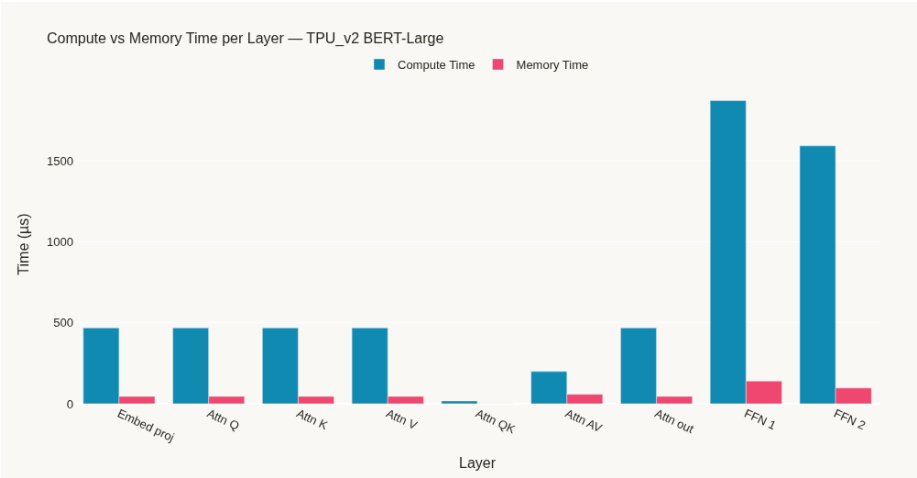

Figure 22: Compute versus memory behavior for BERT-Large on TPU v2.

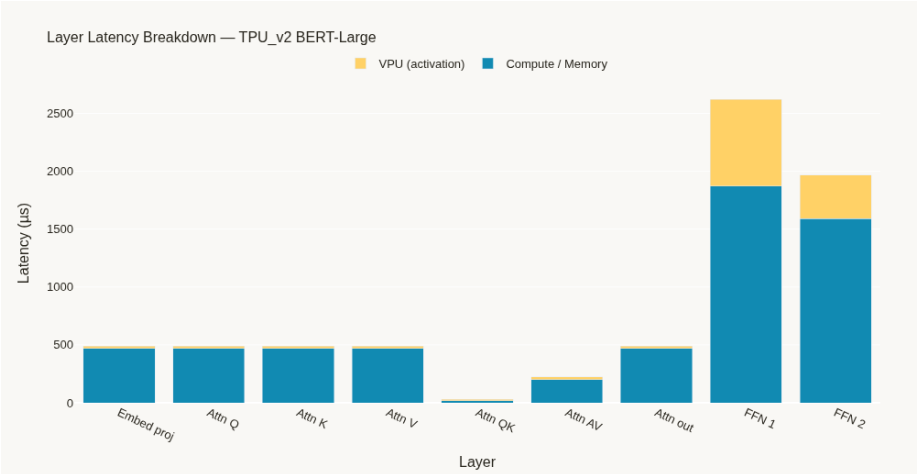

Figure 23: Latency breakdown for BERT-Large on TPU v2.

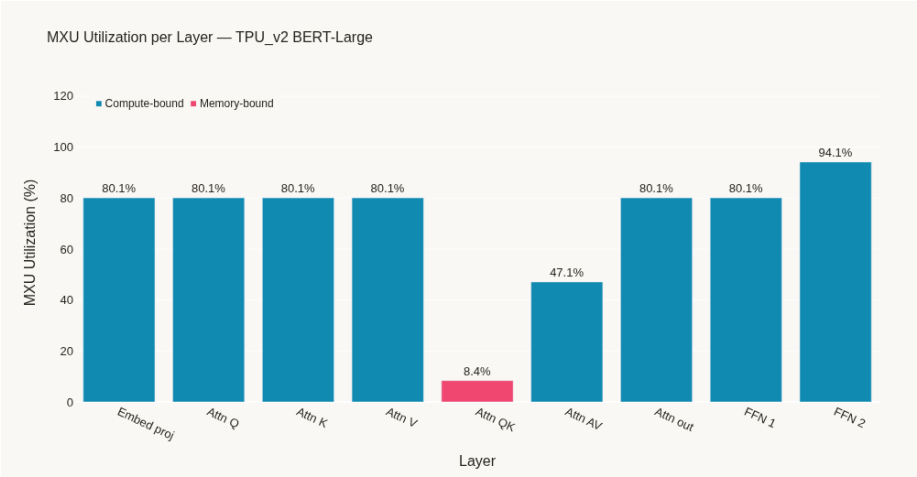

Figure 24: MXU utilization for BERT-Large on TPU v2.

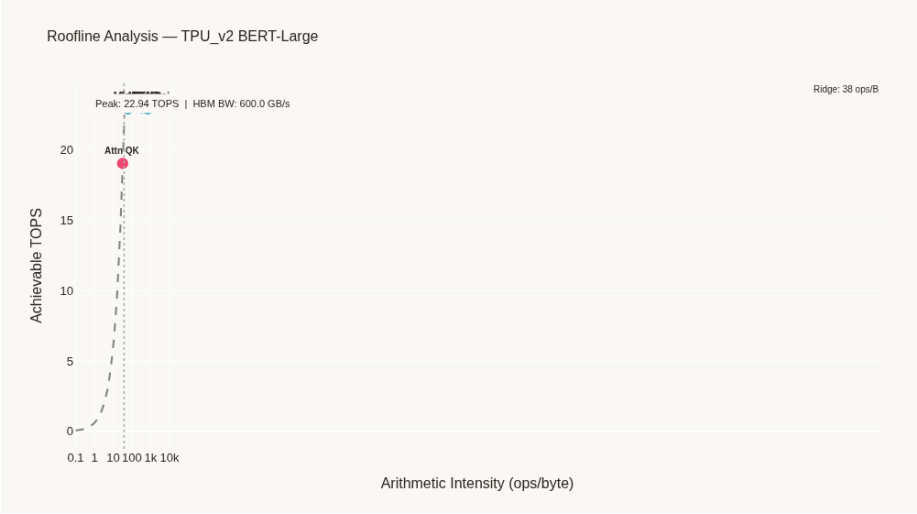

Figure 25: Roofline analysis for BERT-Large on TPU v2.

9.4.2 GPT-2 Medium on TPU v2

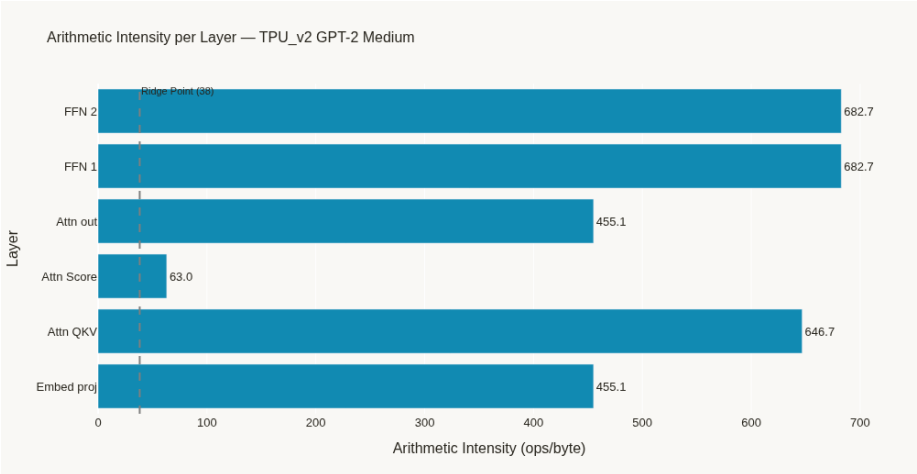

Figure 26: Arithmetic intensity for GPT-2 Medium on TPU v2.

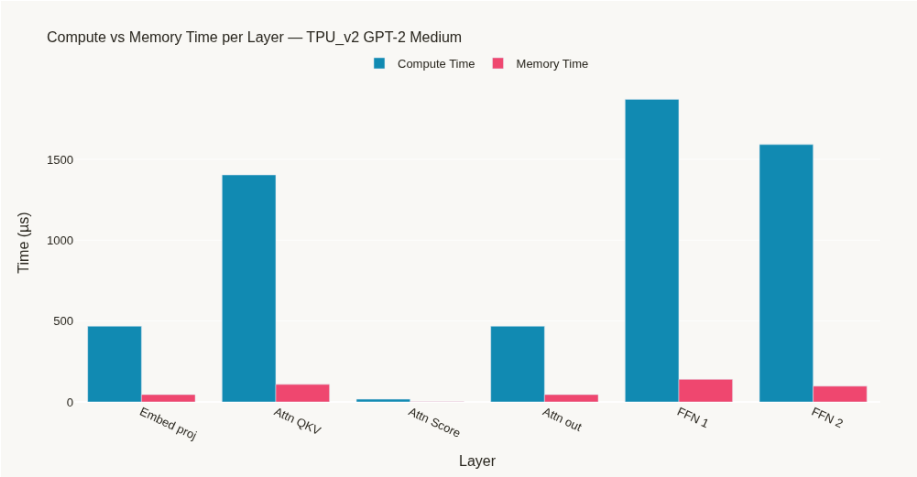

Figure 27: Compute versus memory behavior for GPT-2 Medium on TPU v2.

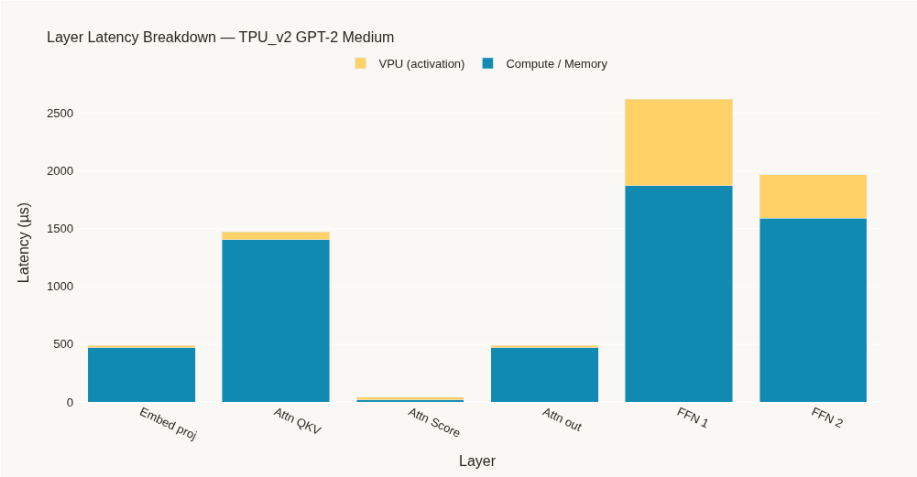

Figure 28: Latency breakdown for GPT-2 Medium on TPU v2.

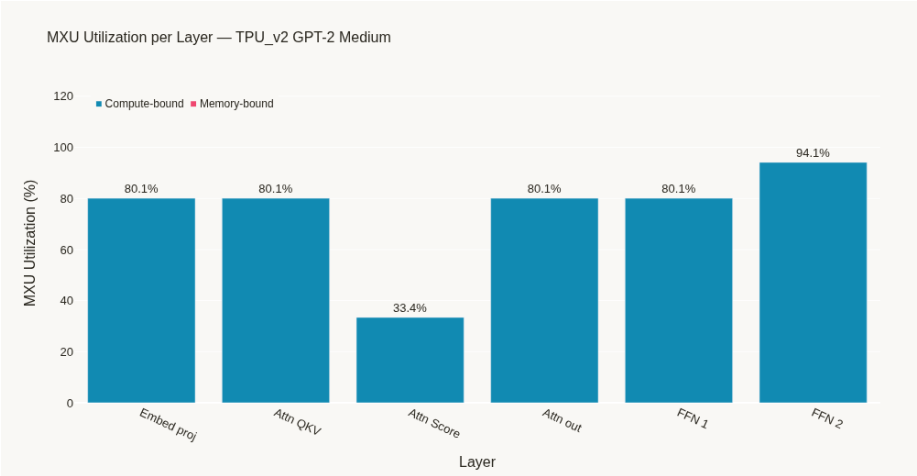

Figure 29: MXU utilization for GPT-2 Medium on TPU v2.

Figure 30: Roofline analysis for GPT-2 Medium on TPU v2.

9.4.3 ResNet-50 on TPU v2

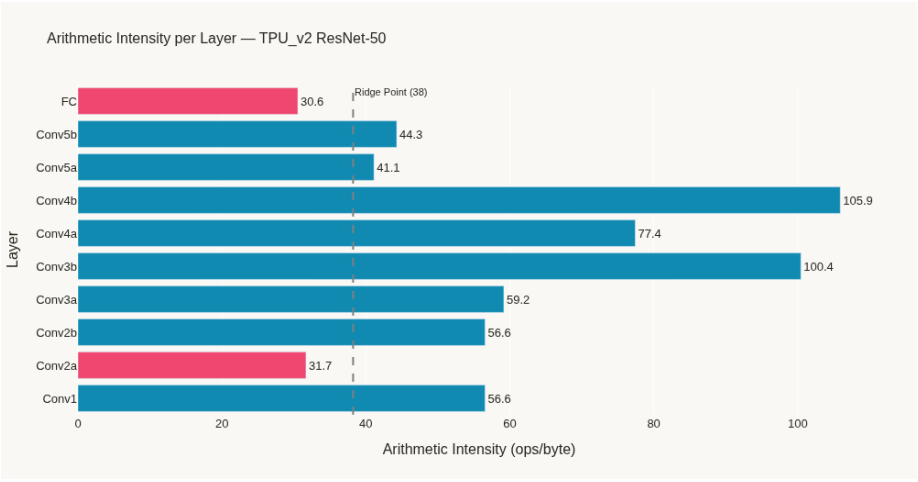

Figure 31: Arithmetic intensity for ResNet-50 on TPU v2.

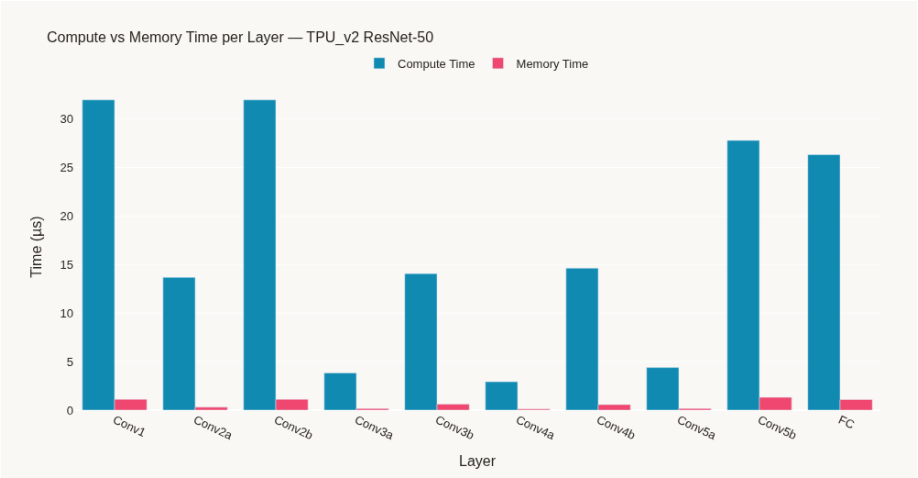

Figure 32: Compute versus memory behavior for ResNet-50 on TPU v2.

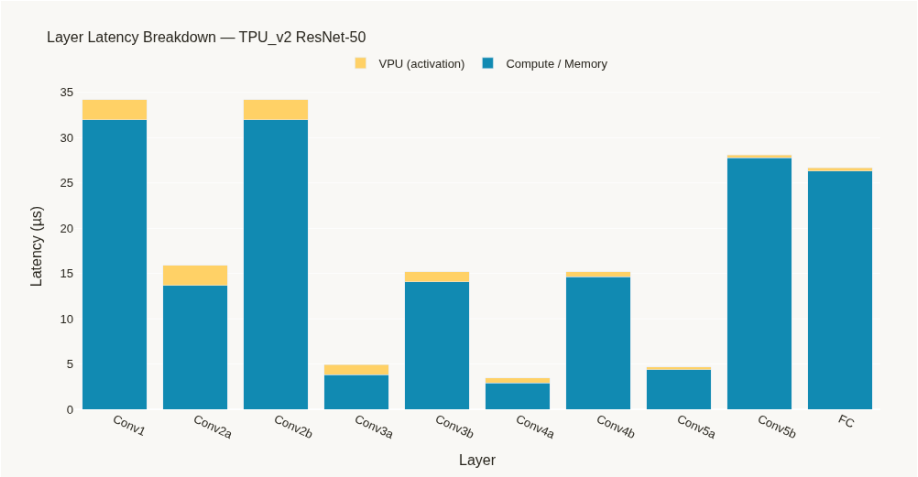

Figure 33: Latency breakdown for ResNet-50 on TPU v2.

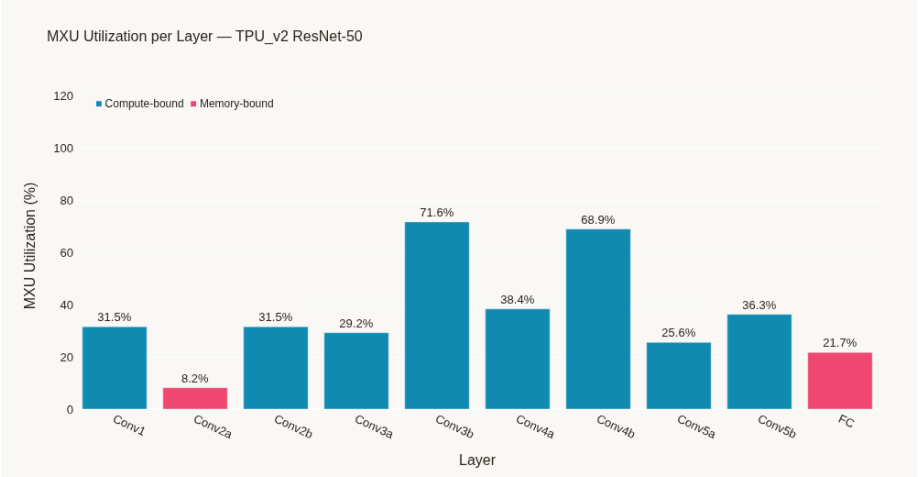

Figure 34: MXU utilization for ResNet-50 on TPU v2.

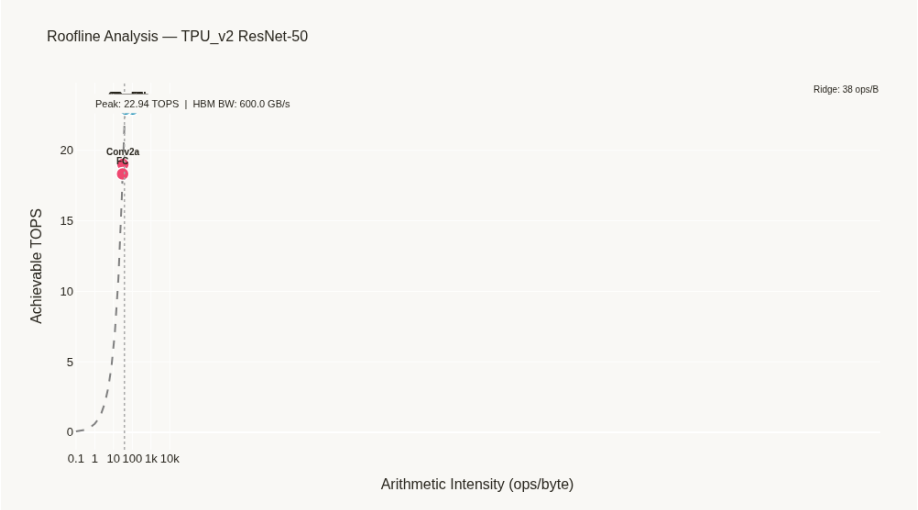

Figure 35: Roofline analysis for ResNet-50 on TPU v2.

9.5 TPU v4 Results

9.5.1 BERT-Large on TPU v4

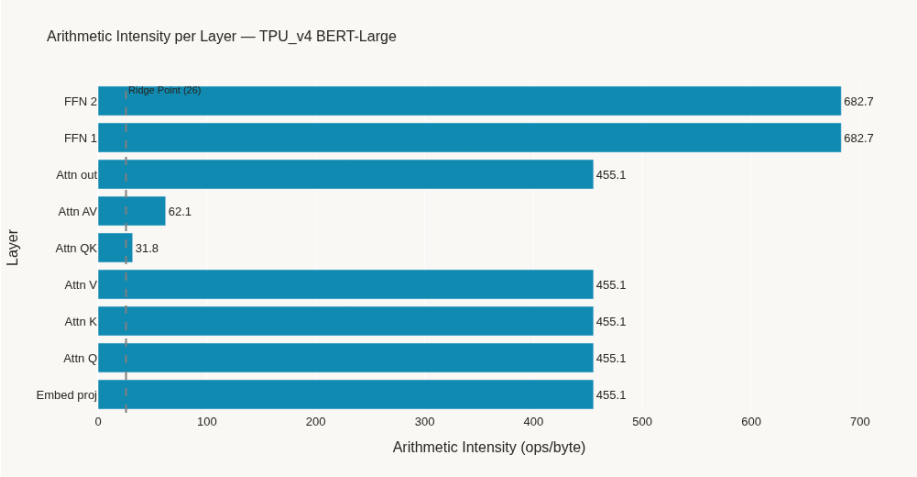

Figure 36: Arithmetic intensity for BERT-Large on TPU v4.

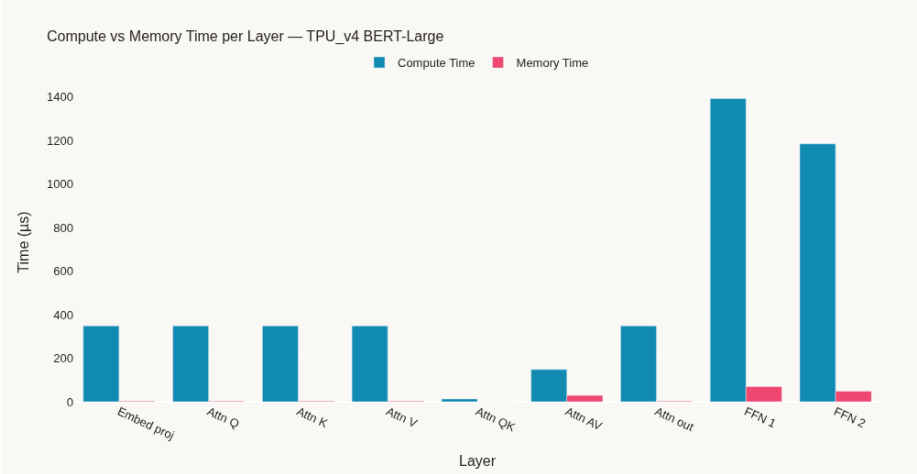

Figure 37: Compute versus memory behavior for BERT-Large on TPU v4.

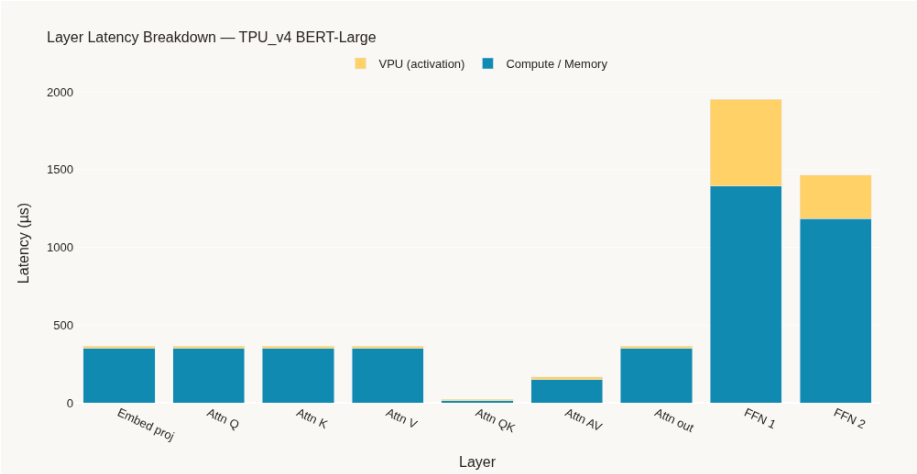

Figure 38: Latency breakdown for BERT-Large on TPU v4.

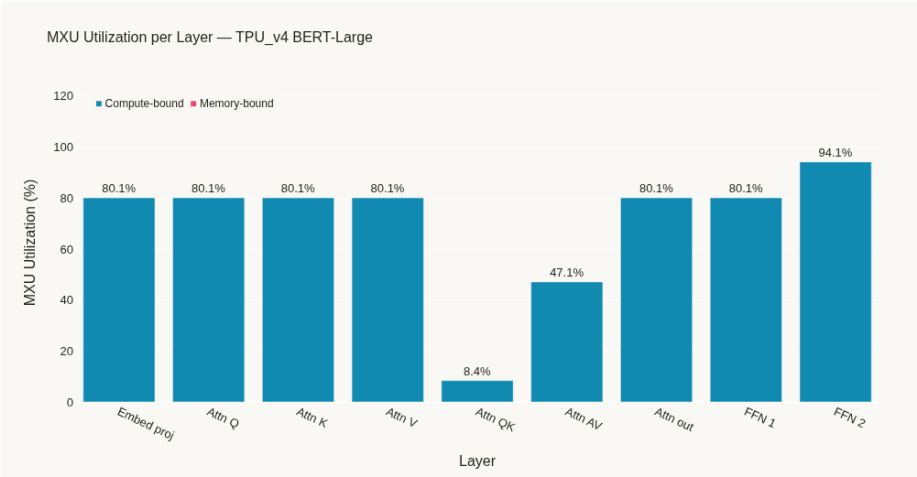

Figure 39: MXU utilization for BERT-Large on TPU v4.

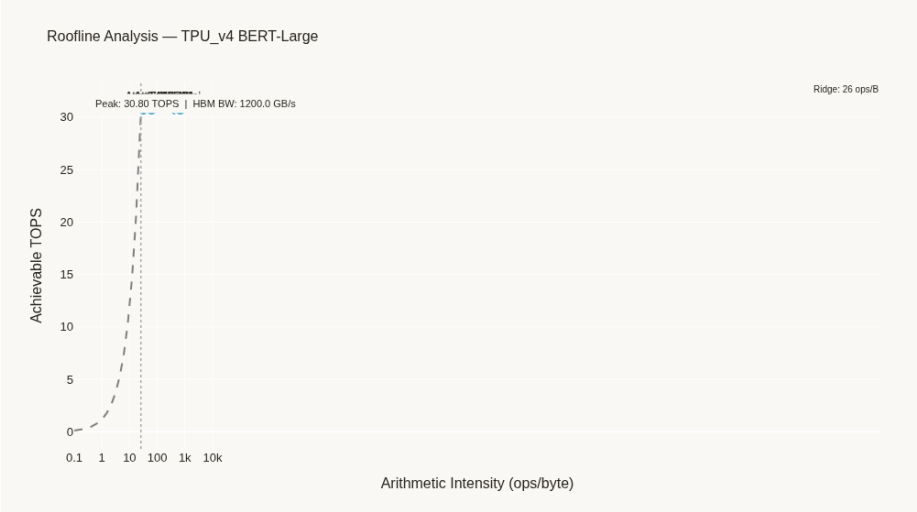

Figure 40: Roofline analysis for BERT-Large on TPU v4.

9.5.2 GPT-2 Medium on TPU v4

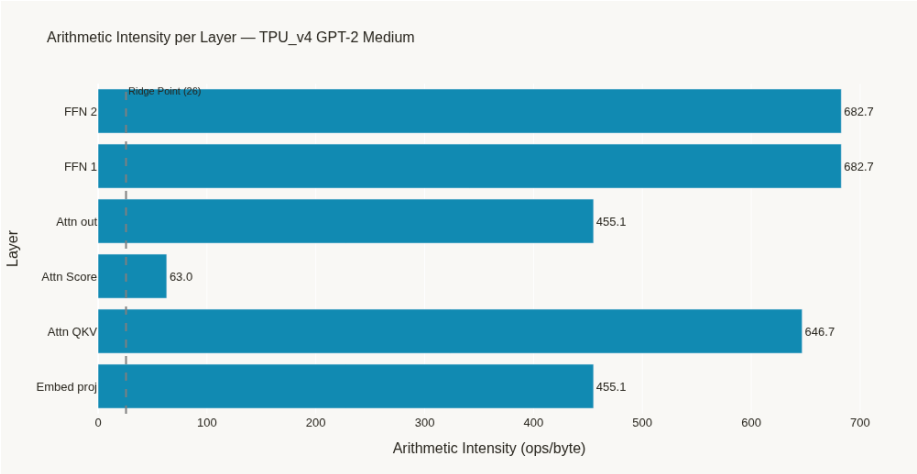

Figure 41: Arithmetic intensity for GPT-2 Medium on TPU v4.

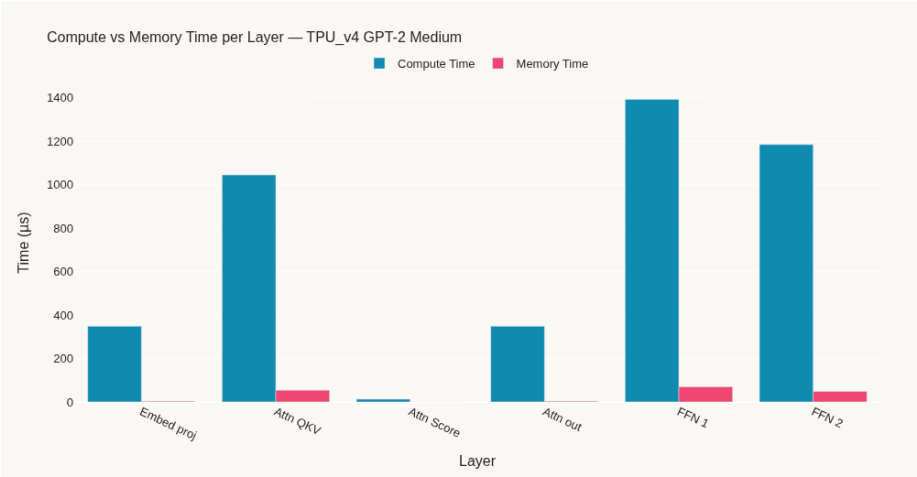

Figure 42: Compute versus memory behavior for GPT-2 Medium on TPU v4.

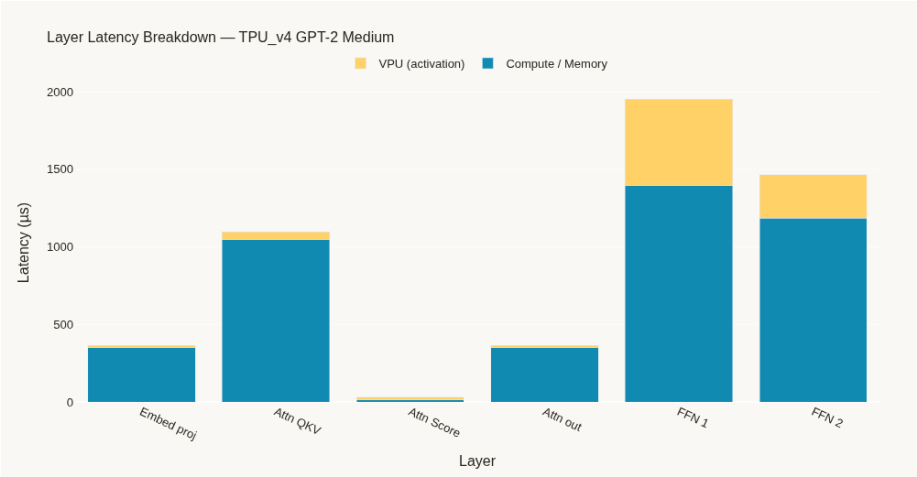

Figure 43: Latency breakdown for GPT-2 Medium on TPU v4.

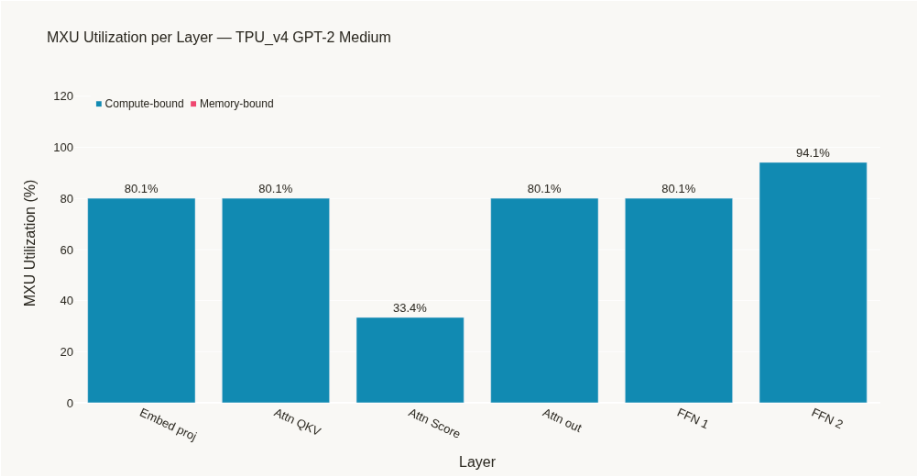

Figure 44: MXU utilization for GPT-2 Medium on TPU v4.

Figure 45: Roofline analysis for GPT-2 Medium on TPU v4.

9.5.3 ResNet-50 on TPU v4

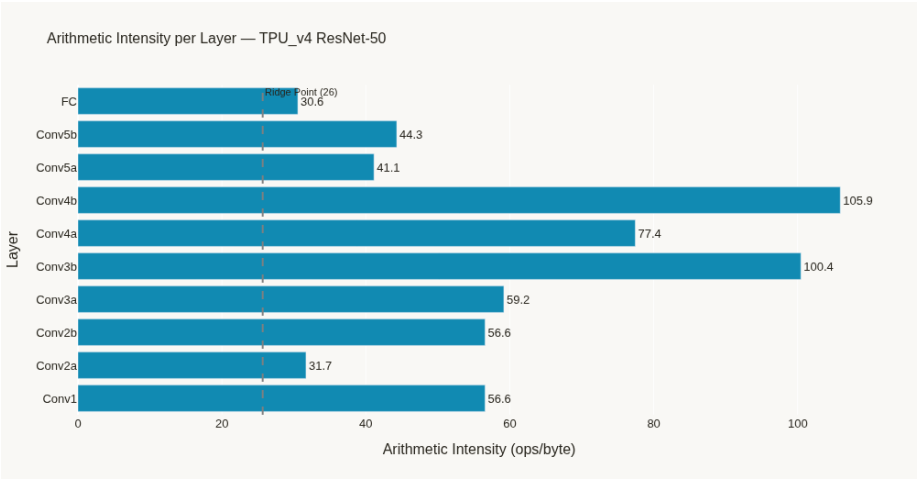

Figure 46: Arithmetic intensity for ResNet-50 on TPU v4.

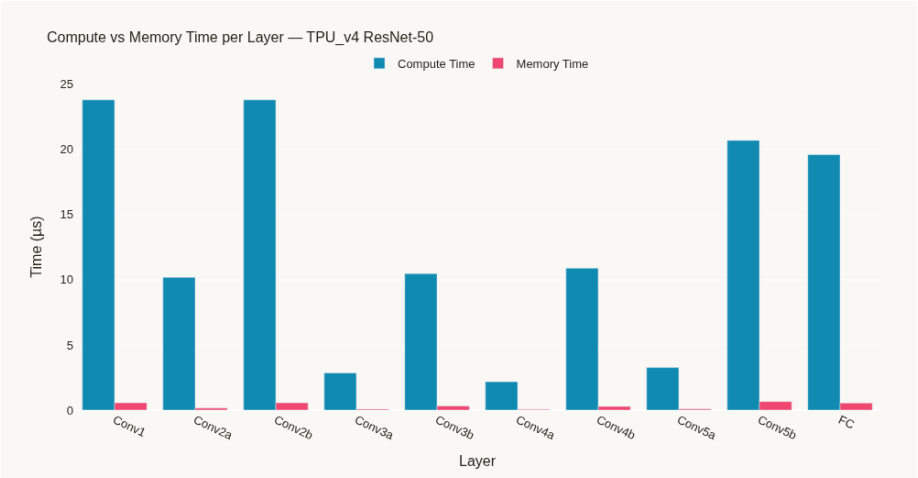

Figure 47: Compute versus memory behavior for ResNet-50 on TPU v4.

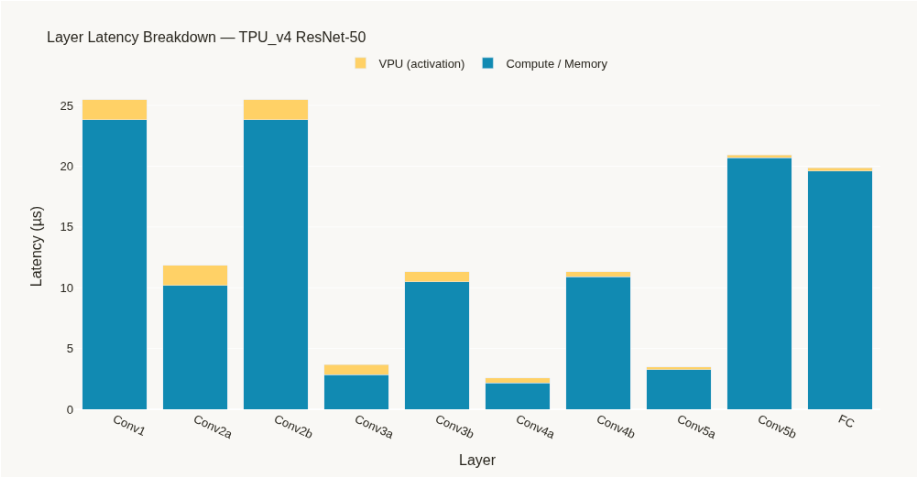

Figure 48: Latency breakdown for ResNet-50 on TPU v4.

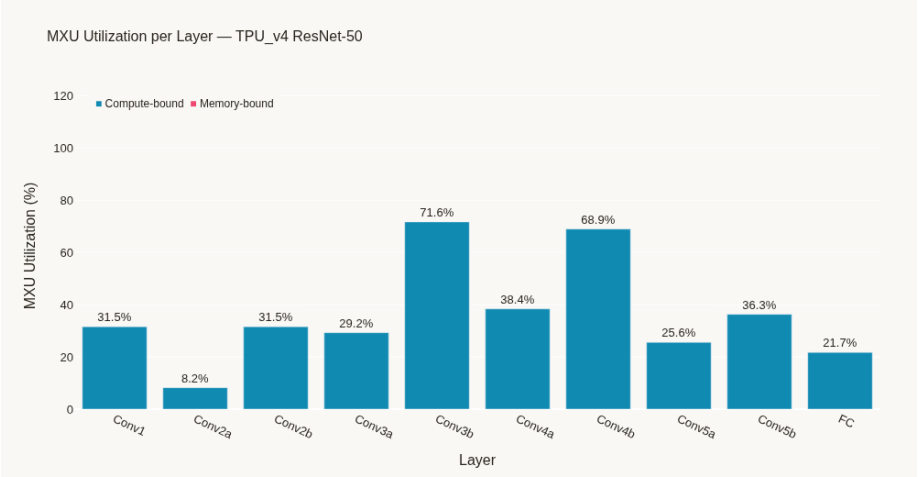

Figure 49: MXU utilization for ResNet-50 on TPU v4.

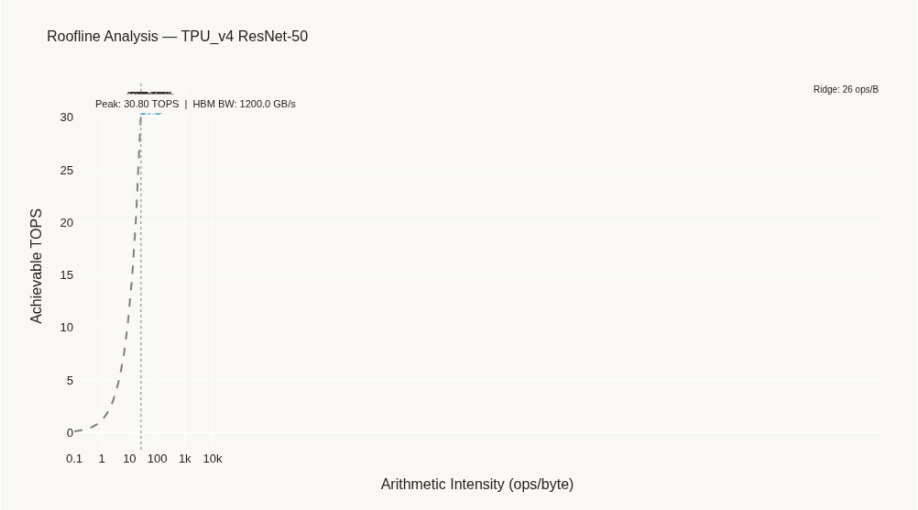

Figure 50: Roofline analysis for ResNet-50 on TPU v4.

9.6 Discussion

The result plots support architectural analysis from multiple viewpoints. Arithmetic intensity helps understand workload characteristics, compute-versus-memory plots highlight bottlenecks, latency breakdown identifies time contributors, MXU utilization indicates how effectively the compute array is used, and roofline plots summarize performance limits visually.

The combined comparison plot is useful for observing high-level trends across TPU generations and workloads. Such a view can help identify which configurations are better suited for transformer workloads and which are more favorable for convolutional models.

10 Advantages and Applications

10.1 Advantages

- Provides practical understanding of TPU-style accelerator concepts
- Allows experimentation without requiring real TPU hardware
- Supports multiple hardware presets and multiple workloads
- Generates reports and graph-based analysis
- Helps identify performance bottlenecks in a structured way
- Suitable for academic learning and project demonstrations

10.2 Applications

- Computer architecture education
- ML systems coursework
- Accelerator performance analysis
- Design-space exploration
- Comparative workload studies
- Research prototyping

11 Conclusion

This project presents a beginner-friendly yet meaningful TPU-inspired architectural simulator for analyzing deep learning workloads. By combining configurable hardware presets, workload descriptions, simulation logic, CSV reporting, and result visualization, the system provides a practical way to study accelerator behavior.

The project is valuable because it turns theoretical architecture concepts into an executable analysis workflow. It also provides a strong foundation for future research and educational work in machine learning systems, accelerator design, and performance modeling.

References

[1] Google Cloud Blog, *An in-depth look at Google's first Tensor Processing Unit (TPU)*. Available at: https://cloud.google.com/blog/products/ai-machine-learning/an-in-depth-look-at-googles-first-tensor-processing-unit-tpu

[2] Google Cloud Documentation, *TPU architecture*. Available at: https://docs.cloud.google.com/tpu/docs/system-architecture-tpu-vm

[3] SCALE-Sim, *Systolic Array Accelerator Simulator*. Available at: https://github.com/ARM-software/SCALE-Sim